

QCM – Technologies Web & Mobile Avancé (révision)

Partie I – REST, RESTful et GraphQL (1 à 20)

1. Une API est qualifiée de RESTful lorsqu'elle respecte :

- A. L'utilisation obligatoire de JSON
- B. Les contraintes architecturales définies par Roy Fielding (stateless, uniform interface, cacheability...)
- C. L'usage exclusif de SOAP
- D. L'utilisation de GraphQL comme standard

2. Le principe REST "stateless" signifie que :

- A. Le serveur conserve l'état de session client
- B. Chaque requête contient toutes les informations nécessaires à son traitement sans dépendance serveur
- C. Les données doivent être mises en cache systématiquement
- D. Les requêtes doivent être synchrones

3. Une opération HTTP est dite idempotente lorsqu'elle :

- A. Change toujours l'état du serveur
- B. Produit un résultat identique après plusieurs exécutions identiques
- C. Est toujours exécutée une seule fois
- D. Est asynchrone

4. Parmi les méthodes suivantes, laquelle est strictement idempotente par conception ?

- A. POST
- B. PATCH
- C. PUT
- D. CONNECT

5. La méthode POST est principalement utilisée pour :

- A. Récupérer des données
- B. Créer une nouvelle ressource sur le serveur
- C. Remplacer totalement une ressource existante
- D. Mettre en cache une réponse

6. PATCH est utilisée dans une API REST pour :

- A. Remplacer complètement une ressource
- B. Appliquer une modification partielle à une ressource
- C. Supprimer une ressource
- D. Lister les ressources

7. Le principal problème que GraphQL cherche à résoudre est :

- A. La consommation CPU élevée
- B. Le sur-fetching et sous-fetching des données dans les API REST
- C. La lenteur des bases de données
- D. Le stockage local des données

8. Dans une architecture GraphQL, l'accès aux données se fait généralement via :

- A. Plusieurs endpoints REST
- B. Un endpoint unique HTTP
- C. Un socket TCP dédié
- D. Un service FTP

9. Le schema GraphQL est utilisé pour :

- A. Définir les tables SQL
- B. Décrire les types de données et leurs relations accessibles à l'API
- C. Gérer le cache HTTP
- D. Configurer le load balancing

10. Une contrainte fondamentale de REST est :

- A. Stateful architecture
- B. Stateless architecture
- C. Couplage fort entre client et serveur
- D. Utilisation obligatoire de XML

11. La méthode HTTP GET est considérée comme :

- A. Non idempotente
- B. Idempotente et sans effet de bord
- C. Toujours destructive
- D. Asynchrone uniquement

12. JWT (JSON Web Token) est principalement utilisé pour :

- A. Le stockage local des images
- B. L'authentification stateless entre client et serveur
- C. Le routage réseau
- D. Le rendu UI

13. REST repose sur quel protocole principal ?

- A. FTP
- B. HTTP

- C. SMTP
- D. UDP

14. GraphQL permet au client de :

- A. Définir dynamiquement les données qu'il souhaite recevoir
- B. Modifier directement la base de données
- C. Remplacer HTTP
- D. Accéder uniquement à des données fixes

15. Une ressource REST est identifiée de manière unique par :

- A. Une classe Java
- B. Une URL (URI)
- C. Un objet mémoire
- D. Une table SQL

16. Le principe "Uniform Interface" signifie :

- A. Une interface utilisateur unique
- B. Une standardisation des interactions client-serveur
- C. Une base de données unique
- D. Un seul serveur backend

17. Cacheability en REST signifie :

- A. Les réponses ne peuvent jamais être mises en cache
- B. Les réponses peuvent être stockées pour améliorer les performances
- C. Les données sont toujours volatiles
- D. Le serveur doit être stateful

18. Layered system implique :

- A. Une architecture en couches indépendantes
- B. Une UI multi-écrans
- C. Un seul serveur monolithique
- D. Une base de données unique

19. REST utilise principalement :

- A. Les verbes HTTP standard
- B. Les sockets TCP
- C. Les fichiers locaux
- D. Les threads système

20. GraphQL améliore REST en :

- A. Forçant plus de requêtes réseau
- B. Permettant une récupération optimisée des données
- C. Supprimant HTTP
- D. Rendant les API statiques

Partie II – Flutter & Dart (21 à 40)

21. Flutter est basé sur le langage :

- A. Java
- B. Dart
- C. Kotlin
- D. Swift

22. Dart est un langage :

- A. Orienté objet compilé
- B. Exclusivement fonctionnel
- C. SQL natif
- D. Assembleur

23. Un StatelessWidget est :

- A. Un widget avec état dynamique
- B. Un widget immuable sans état interne
- C. Une base de données UI
- D. Un service backend

24. Un StatefulWidget permet :

- A. Aucune modification UI
- B. Une gestion dynamique de l'état
- C. Un accès direct au réseau
- D. Une compilation native

25. Flutter utilise le moteur de rendu :

- A. Skia
- B. Unity
- C. DirectX
- D. OpenGL ES uniquement

26. async/await en Dart permet :

- A. De bloquer l'application
- B. De gérer des opérations asynchrones proprement

- C. De compiler plus vite
- D. De créer des widgets

27. Une Future en Dart représente :

- A. Une valeur déjà disponible
- B. Une valeur disponible ultérieurement
- C. Un thread système
- D. Une UI component

28. Stream en Dart représente :

- A. Un flux de données asynchrone
- B. Une base de données
- C. Une UI
- D. Un serveur

29. setState() sert à :

- A. Modifier la base de données
- B. Rebuild l'interface utilisateur
- C. Créer une API
- D. Gérer le routing

30. Flutter est une technologie :

- A. Mobile uniquement Android
- B. Cross-platform
- C. Backend uniquement
- D. Desktop uniquement

31. Hot Reload permet :

- A. Redémarrage complet du système
- B. Mise à jour instantanée de l'UI
- C. Compilation lente
- D. Suppression du cache

32. Widget Flutter est :

- A. Une donnée backend
- B. Un élément UI
- C. Une API
- D. Une base de données

33. Dart supporte :

- A. Programmation orientée objet
- B. SQL runtime
- C. HTML natif
- D. Bash scripting

34. Flutter est développé par :

- A. Apple
- B. Google
- C. Microsoft
- D. Meta

35. BuildContext sert à :

- A. Accéder à la structure de l'arbre UI
- B. Gérer la base de données
- C. Créer une API REST
- D. Gérer le réseau

36. Widget tree représente :

- A. Une base SQL
- B. La hiérarchie UI
- C. Un serveur backend
- D. Une API

37. Dart event loop permet :

- A. Multithreading classique
- B. Gestion des tâches asynchrones
- C. UI rendering uniquement
- D. DB indexing

38. Flutter compile vers :

- A. JavaScript uniquement
- B. Code natif
- C. PHP
- D. SQL

39. Provider en Flutter sert à :

- A. Gérer l'état
- B. Créer des APIs
- C. Gérer la DB
- D. Compiler Dart

40. Flutter est utilisé pour :

- A. Applications mobiles multiplateformes
- B. Serveurs backend uniquement
- C. Bases de données
- D. OS development

Partie III – Kafka & RabbitMQ (41 à 55)

41. Kafka est un :

- A. SGBD relationnel
- B. Broker de streaming distribué
- C. Framework UI
- D. OS

42. RabbitMQ utilise le protocole :

- A. AMQP
- B. HTTP
- C. FTP
- D. SMTP

43. Kafka organise les données en :

- A. Tables
- B. Topics
- C. Pages
- D. Fichiers UI

44. Une partition Kafka permet :

- A. UI rendering
- B. Scalabilité et parallélisme
- C. CSS management
- D. DB locking

45. Offset Kafka représente :

- A. Une erreur
- B. La position d'un message dans un log
- C. Un widget UI
- D. Une API

46. Producer Kafka :

- A. Consomme messages

- B. Publie messages
- C. UI rendering
- D. DB access

47. Consumer Kafka :

- A. Lit les messages
- B. Crée UI
- C. Gère DB
- D. Compile code

48. Kafka conserve les messages :

- A. Temporairement uniquement
- B. Persistants selon configuration
- C. UI only
- D. RAM only

49. RabbitMQ Exchange sert à :

- A. Stocker messages
- B. Router messages
- C. UI rendering
- D. DB indexing

50. Fanout Exchange :

- A. Routing ciblé
- B. Broadcast à toutes les queues
- C. Suppression messages
- D. Encryption

51. Direct Exchange :

- A. Broadcast global
- B. Routing par clé exacte
- C. UI rendering
- D. DB replication

52. Kafka est optimisé pour :

- A. Big Data streaming
- B. UI rendering
- C. CSS
- D. HTML

53. RabbitMQ est :

- A. Messaging system
- B. UI framework
- C. OS
- D. DB

54. Kafka architecture est :

- A. Centralisée
- B. Distribuée
- C. UI only
- D. Single node

55. Producer Kafka :

- A. Lit messages
- B. Envoie messages
- C. UI
- D. DB

Partie IV – Microservices & Eureka (56 à 65)

56. Eureka sert à :

- A. UI
- B. Service discovery
- C. DB
- D. Cache

57. Microservices signifient :

- A. Monolithe
- B. Services indépendants
- C. UI
- D. DB

58. API Gateway sert à :

- A. Routing centralisé
- B. UI
- C. DB
- D. OS

59. Service registry contient :

- A. UI
- B. Services disponibles

- C. CSS
- D. DB

60. Eureka client :

- A. S'enregistre
- B. UI
- C. DB
- D. Cache

61. Microservices communiquent via :

- A. HTTP/REST
- B. UI
- C. CSS
- D. DB

62. Avantage microservices :

- A. Scalabilité
- B. Couplage fort
- C. UI lente
- D. DB unique

63. Gateway permet :

- A. Routing
- B. UI
- C. DB
- D. OS

64. Load balancing permet :

- A. UI
- B. Répartition charge
- C. DB
- D. CSS

65. Microservices favorisent :

- A. Indépendance
- B. Monolithe
- C. UI only
- D. DB unique

66. Event-driven architecture repose sur :

- A. Events
- B. UI
- C. DB
- D. CPU

67. Tolérance aux pannes signifie :

- A. Crash system
- B. Continuer fonctionnement
- C. UI fail
- D. DB reset

68. Scalabilité signifie :

- A. Réduction
- B. Support charge croissante
- C. UI scaling
- D. CSS

69. Cloud-native utilise :

- A. Microservices
- B. Monolithes
- C. Desktop apps
- D. Local DB

70. Couplage faible signifie :

- A. Forte dépendance
- B. Indépendance des services
- C. UI dependency
- D. DB dependency